Lab 03: The Post-API Age

INFO 4617/5617 · Week 3 · Chapter 3, The Post-API Age

Today we code together. Each step below says what we're building and why, and its code cell holds only the comments that frame it: we write the code live, in class. Nothing here is turned in. After each step, a 💬 **Discuss** box holds questions to talk over before the next one; click it to open it. A ⏱️ **Microsprint** is a few minutes' task for you alone or with a neighbour, ending in something to share.

The chapter argues that access to web data is shrinking through enclosure, exemption, and erosion. Today we check that by measurement: what eight platforms say to a request with no credentials, who is really refusing when one says no, how three retired research endpoints fail, what the Wayback Machine remembers of them, and how differently six servers of one federated network answer the same question.

The case: the chapter's own audit ("Auditing Access Yourself"), its eight endpoints and three retired ones, run again today. Your table will differ from the chapter's and from your neighbour's; when and where you ran it is part of the result.

Before class: open this notebook in the `webdata` environment (chapter 1), and have your email address ready for the User-Agent. No keys and no accounts: every request today is anonymous on purpose.

On Friday, run every cell above where we stopped (**Run → Run All Above Selected Cell**).

> **Instructor key.** Every cell is filled in and was run against the live sites on 2026-10-21 at 17:34 UTC, from a cloud machine, except Step 5's two cells, which are tagged `not-run-live`: the Wayback Machine answered this machine with 429 every time. Run the key with `labs/tools/run_key.py` on a laptop to fill them in. An audit is a dated observation, so a rerun will differ in places: Step 2's 429 from Wikipedia came from the cloud machine's shared address, and a laptop on campus will probably see a 200. The lab follows the textbook at commit d9306b7.

Step 0 — Set up (5 minutes)

Import `requests`, `pandas`, `matplotlib`, and `time`. Then define `HEADERS`, the User-Agent every request will send. Start from chapter 2's form, `WebDataScience/1.0 (your-email@colorado.edu)`, and add the one part Wikimedia's User-Agent policy asks for at the end: the library and its version, `python-requests/` and `requests.__version__`. Step 3 tests whether the name matters.

Last, record when we ran the audit, in UTC, with `pd.Timestamp.now()`. The chapter's outputs "were recorded on 28 August 2026, and yours will differ"; as its section on openness puts it, "an access matrix run once is a snapshot, while the same matrix run every semester is a record of enclosure happening."

```
# Import requests, pandas (as pd), matplotlib.pyplot (as plt), and time
# Define HEADERS: a dict whose "User-Agent" is "WebDataScience/1.0 (your-email@colorado.edu)",
#   a space, then "python-requests/" and requests.__version__
# Define AUDIT_DATE: the time now in UTC, as text like "2026-10-21 17:00 UTC"; print it and
 ↪  HEADERS
import requests
import pandas as pd
import matplotlib.pyplot as plt
import time

HEADERS = {"User-Agent": f"WebDataScience/1.0 (your-email@colorado.edu)
 ↪  python-requests/{requests.__version__}"}
AUDIT_DATE = pd.Timestamp.now(tz="UTC").strftime("%Y-%m-%d %H:%M UTC")
print(HEADERS)
print("Audit run", AUDIT_DATE)
```

```
{'User-Agent': 'WebDataScience/1.0 (your-email@colorado.edu) python-requests/2.34.2'}
Audit run 2026-10-21 17:34 UTC
```

💬 Discuss

- In week 2 we put our email address in a User-Agent. Who reads it on the other end today, and what could they do with it?
- Why is a measurement of access without its date worse than no measurement at all?
- Week 1 saved pageviews to a CSV because the data may change. What would we save from today's audit so that next semester's class could compare?

Step 1 — One request, read the whole answer (7 minutes)

Before probing eight hosts, look closely at two. Ask Reddit's public JSON endpoint and Hacker News's API for their front pages, and for each print three parts of the `Response` that week 1 introduced: the status code, the `content-type` header, and the start of the body. Pass `timeout=20`, so that a host that never answers can't stall the notebook. The chapter's point (in "An Access Matrix") is that what a host sends back is as informative as the code it sends: the content type tells you whether you received data at all.

```
# Reddit: https://www.reddit.com/r/python/hot.json?limit=1
# Hacker News: https://hacker-news.firebaseio.com/v0/topstories.json
# Request each once with HEADERS and timeout=20; print its status code and content-type header,
#   and under them the first 80 characters of the body, with repr()
FRONT_PAGES = {"Reddit": "https://www.reddit.com/r/python/hot.json?limit=1",
               "Hacker News": "https://hacker-news.firebaseio.com/v0/topstories.json"}
for name, url in FRONT_PAGES.items():
    response = requests.get(url, headers=HEADERS, timeout=20)
    print(name, response.status_code, response.headers.get("content-type"))
    print("   ", repr(response.text[:80]))
```

```
Reddit 403 text/html
    '<body class=theme-beta><div><style>.theme-light,:root{--rem360:22.5rem;--rem320:'
Hacker News 200 application/json; charset=utf-8
    '[50180873,50182987,50181114,50181136,50181175,50183727,50177916,50182526,5016204'
```

💬 Discuss

- Both hosts answered. Which one answered the question we asked, and which header told you before you read the body?
- What would `response.json()` do with Reddit's answer, and what would the error message make you think was wrong?
- In week 2, Reddit's `robots.txt` was eight lines that disallow every path. Is today's 403 the same decision, enforced in a different place?

Step 2 — The access matrix (15 minutes)

The chapter's first audit asks eight services one question: if I show up with no credentials, what happens? `TARGETS`, below, holds its eight endpoints, which ask for no key, no account, and no application. Write the chapter's `VERDICTS`, what a status code means for an anonymous request, and its `probe(name, url)`. It returns a row rather than printing, so the rows compose into a table, and it records a request that never completes as a result instead of letting the error end the audit: catch `requests.RequestException`, the parent of every network error. Make the rows a DataFrame, and give its status column pandas' capital-I `Int64` type, which can hold the `None` of a host that never answered. See "An Access Matrix".

```
# Given: the chapter's eight endpoints, from institutions built to share data to platforms built
 ↪  to fence it
TARGETS = {
```

```
    "Wikipedia":   "https://en.wikipedia.org/w/api.php?action=query&meta=siteinfo&format=json",
    "Hacker News": "https://hacker-news.firebaseio.com/v0/topstories.json",
    "arXiv":       "http://export.arxiv.org/api/query?search_query=all:electron&max_results=1",
    "Mastodon":    "https://mastodon.social/api/v1/instance",
    "Bluesky":     "https://public.api.bsky.app/xrpc/app.bsky.actor.getProfile?actor=bsky.app",
    "Reddit":      "https://www.reddit.com/r/python/hot.json?limit=1",
    "X (Twitter)": "https://api.twitter.com/2/tweets/search/recent?query=test",
    "Meta Graph":  "https://graph.facebook.com/v20.0/me",
}
```

```
# Define VERDICTS: 200 is "open", 401 and 403 are "gated", 429 is "rate-limited"
# Define probe(name, url): request url with HEADERS and timeout=20; on requests.RequestException
 ↪  return
#   status None, verdict "unreachable", and the error's type name as content_type. Otherwise
 ↪  return
#   api, status, verdict (VERDICTS.get, else "other"), content_type (before any ";"), rate_limit
 ↪  (x-ratelimit-limit)
# Probe every target into a DataFrame called access, make status Int64, and print it without the
 ↪  index
VERDICTS = {200: "open", 401: "gated", 403: "gated", 429: "rate-limited"}


def probe(name, url):
    """Ask one endpoint for data without credentials and describe the answer."""
    try:
        response = requests.get(url, headers=HEADERS, timeout=20)
    except requests.RequestException as error:
        return {"api": name, "status": None, "verdict": "unreachable",
                "content_type": type(error).__name__, "rate_limit": None}

    verdict = VERDICTS.get(response.status_code, "other")

    return {
        "api": name,
        "status": response.status_code,
        "verdict": verdict,
        "content_type": response.headers.get("content-type", "").split(";")[0],
        "rate_limit": response.headers.get("x-ratelimit-limit"),
    }


access = pd.DataFrame([probe(name, url) for name, url in TARGETS.items()])
access["status"] = access["status"].astype("Int64")
print(access.to_string(index=False))
```

```
        api  status      verdict             content_type rate_limit
  Wikipedia     429 rate-limited                text/html        NaN
Hacker News     200         open         application/json        NaN
      arXiv     200         open     application/atom+xml        NaN
   Mastodon     200         open         application/json        300
    Bluesky     200         open         application/json        NaN
     Reddit     403        gated                text/html        NaN
X (Twitter)     401        gated application/problem+json        NaN
 Meta Graph     400        other         application/json        NaN
```

⏱️ Microsprint (5 minutes, alone or in pairs)

Do: put the statuses the chapter recorded on 28 August 2026 in a dictionary, and print the rows of `access` whose status differs from the chapter's. **Share:** which rows changed, if any, and for each, whether you think the platform changed or something on our side did.

```
# Microsprint: CHAPTER, the chapter's statuses: 200 for Wikipedia, Hacker News, arXiv, Mastodon,
 ↪  and Bluesky;
#   403 for Reddit, 401 for X (Twitter), 400 for Meta Graph
# Add a chapter_status column with .map(CHAPTER); print the rows where status differs from it
#   (fillna(0) first, so that a host that never answered counts as a change)
CHAPTER = {"Wikipedia": 200, "Hacker News": 200, "arXiv": 200, "Mastodon": 200, "Bluesky": 200,
           "Reddit": 403, "X (Twitter)": 401, "Meta Graph": 400}
access["chapter_status"] = access["api"].map(CHAPTER)
changed = access[access["status"].fillna(0) != access["chapter_status"]]
print(f"{len(changed)} of {len(access)} rows differ from the chapter's table")
print(changed[["api", "chapter_status", "status", "verdict",
 ↪  "content_type"]].to_string(index=False))
```

```
1 of 8 rows differ from the chapter's table
      api  chapter_status  status      verdict content_type
Wikipedia             200     429 rate-limited    text/html
```

💬 Discuss

- Read the rows, not the totals. What do the open services have in common in who runs them, and what do the gated ones have in common?
- Mastodon's is the only row with a `rate_limit`. What does a budget stated up front let a careful client do that a 429 doesn't?
- Week 2 paused a second between requests to one site. Why can `probe()` send its eight requests without pausing, and when would that stop being true?

Step 3 — Read the failure, not just the status code (13 minutes)

Before writing down ”Reddit blocks unauthenticated access,” rule out a more boring explanation: that something between us and Reddit blocked it. The chapter's `diagnose(url)` prints who actually answered: the `server` header (the software that sent the response), the `via` header (the proxies and caches it passed through on the way), the content type, and the start of the body, because a refusal usually says who is refusing. Run it on Reddit's row. See ”Reading the Failure, Not Just the Status Code”.

```
# Define diagnose(url): request url with HEADERS and timeout=20; print the status code,
#   the server and via headers, the content-type, and the first 120 characters of the body (with
 ↪  !r)
# Diagnose Reddit's endpoint, TARGETS["Reddit"]
def diagnose(url):
    """Show who actually answered, so a refusal can be attributed correctly."""
    response = requests.get(url, headers=HEADERS, timeout=20)
    print(f"status       {response.status_code}")
    print(f"server       {response.headers.get('server')}")
    print(f"via          {response.headers.get('via')}")
    print(f"content-type {response.headers.get('content-type')}")
    print(f"body         {response.text[:120]!r}")


diagnose(TARGETS["Reddit"])
```

```
status       403
server       snooserv
via          1.1 varnish
content-type text/html
body         '<body class=theme-beta><div><style>.theme-light,:root{--rem360:22.5rem;--rem320:20
 ↪  rem;--rem192:12rem;--rem144:9rem;--rem'
```

⏱️ Microsprint (5 minutes, alone or in pairs)

Do: test the chapter's comment on its `HEADERS`, that "several of these hosts answer anonymous library defaults differently than they answer a named client." Send Wikipedia, Reddit, Hacker News, and Mastodon one request with no `headers` (so `requests` sends its default User-Agent) and one with `HEADERS`, and print the two statuses side by side. **Share:** which hosts answer the two identities differently, and what the body says when one refuses.

```python
# Microsprint: print requests' default User-Agent, requests.utils.default_user_agent()
# For Wikipedia, Reddit, Hacker News, and Mastodon: request TARGETS[name] once with no headers
 ↪  and once
#   with HEADERS (timeout=20, time.sleep(1) after each); print both status codes and the first
#   60 characters of the default request's body
print("default User-Agent:", requests.utils.default_user_agent())
for name in ["Wikipedia", "Reddit", "Hacker News", "Mastodon"]:
    default = requests.get(TARGETS[name], timeout=20)
    time.sleep(1)
    named = requests.get(TARGETS[name], headers=HEADERS, timeout=20)
    time.sleep(1)
    print(f"{name:<12} default {default.status_code}   named {named.status_code}
     ↪  {default.text[:60]!r}")
```

```
default User-Agent: python-requests/2.34.2

Wikipedia    default 403   named 429   'Please set a user-agent and respect our robot policy
 ↪  https:/'

Reddit       default 403   named 403   '<body
 ↪  class=theme-beta><div><style>.theme-light,:root{--rem3'

Hacker News  default 200   named 200
 ↪  '[50180873,50182987,50181114,50181136,50181175,50183727,50177'

Mastodon     default 200   named 200
 ↪  '{"uri":"mastodon.social","title":"Mastodon.social","short_de'
```

💬 Discuss

- `snooserv` is Reddit's own server software. What would you have concluded if the `server` header had named a proxy you didn't choose, or the body had described your own network?
- The chapter's four-outcomes table says neither side can see the other's intent. What does a named User-Agent tell a platform about which row we're in, and what can't it tell?
- Week 2 said that after hiQ and Van Buren, legal risk concentrates at authentication gates and contracts, not at public visibility. Which of today's refusals is a gate in that sense, and which is a block page?

Step 4 — Dead-endpoint forensics (16 minutes)

The chapter's second audit asks three retired research endpoints for data, and classifies how each refuses, because the manner of refusal says whether anything can still be recovered. Write its `autopsy(name, url)`: no HTTP response at all means the host doesn't answer; 401 or 403 means the service is still running, for someone else; 404 means the host is alive but the endpoint was removed;

and 400 means it still parses your request and rejects it without a token. Use `timeout=25`: a host that has been shut down may drop packets rather than refuse, and without a timeout the cell would wait forever. See ”Dead-Endpoint Forensics”.

```
# Given: three retired endpoints that research once depended on
RETIRED = {
    "Pushshift (Reddit archive)":
     ↪  "https://api.pushshift.io/reddit/search/submission/?q=test&size=1",
    "CrowdTangle (Meta)":         "https://api.crowdtangle.com/posts",
    "Twitter API v1.1":
     ↪  "https://api.twitter.com/1.1/statuses/user_timeline.json?screen_name=nasa",
}
```

```
# Define autopsy(name, url): request with HEADERS and timeout=25; on requests.RequestException
 ↪  return
#   status None, cause "host does not answer", and the error's type name as evidence. Otherwise
 ↪  look the
#   status up in a dict of causes (200, 400, 401, 403, 404; anything else "other (<code>)"), and
 ↪  return
#   the first 60 characters of the body, newlines replaced by spaces, as evidence
# Autopsy each of RETIRED: print endpoint, status, and cause on one line, and the evidence under
 ↪  it
def autopsy(name, url):
    """Ask a retired endpoint for data and classify how it refuses."""
    try:
        response = requests.get(url, headers=HEADERS, timeout=25)
    except requests.RequestException as error:
        return {"endpoint": name, "status": None,
                "cause": "host does not answer", "evidence": type(error).__name__}

    causes = {
        200: "still open",
        400: "rejects the request without a token",
        401: "still running, credentials now required",
        403: "still running, credentials now required",
        404: "host alive, endpoint removed",
    }
    return {"endpoint": name, "status": response.status_code,
            "cause": causes.get(response.status_code, f"other ({response.status_code})"),
            "evidence": response.text[:60].replace("\n", " ")}


for name, url in RETIRED.items():
    result = autopsy(name, url)
    print(f"{result['endpoint']:<28} {str(result['status']):<6} {result['cause']}")
    print(f"{'':<28} {'':<6} {result['evidence']}")
```

```
Pushshift (Reddit archive)   403    still running, credentials now required
                                    {"detail":"Not authenticated"}

CrowdTangle (Meta)           None   host does not answer
                                    ProxyError

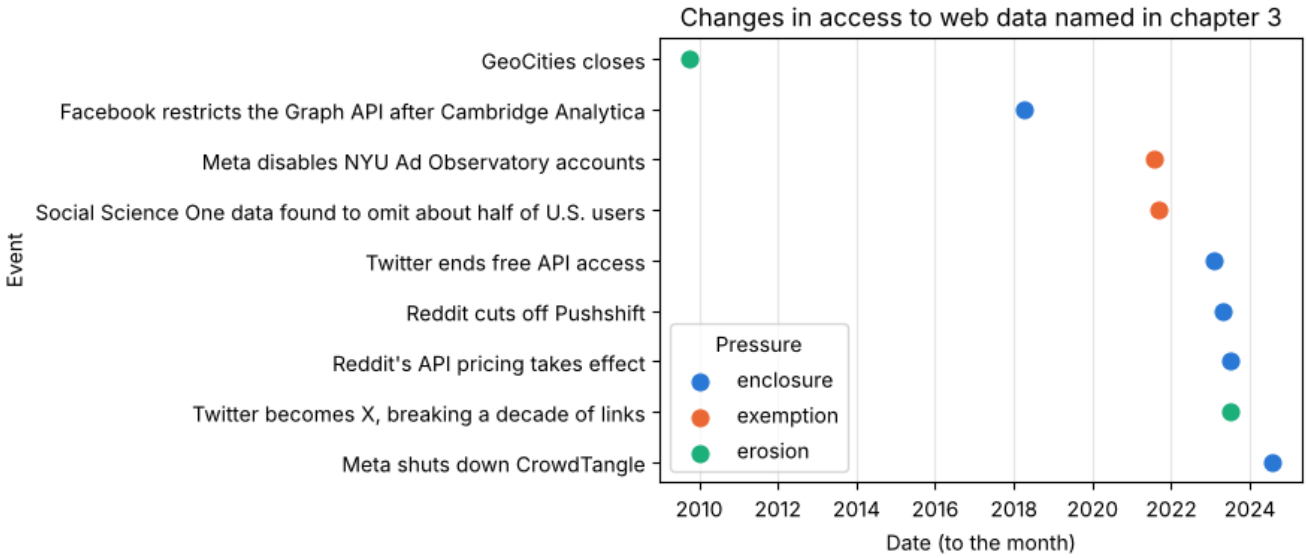
Twitter API v1.1             400    rejects the request without a token
                                    {"errors":[{"code":215,"message":"Bad Authentication data."}
```

⏱️ Microsprint (6 minutes, alone or in pairs)

Do: a short version of Additional Exercise 2. Make a DataFrame of at least five dated events from the chapter, across at least three platforms, each labelled with the pressure that fits it best (enclosure, exemption, or erosion), and plot them as a timeline: one row per event, in date order, coloured by pressure. **Share:** your plot, and the event you found hardest to label.

```
# Microsprint: EVENTS, a list of (date, event, pressure): five or more events from the chapter,
 ↪  from three or more platforms
# Make a DataFrame sorted by date; plot one point per event (x the date, y its row), coloured by
 ↪  pressure,
#   with the events as y tick labels, a legend, a title, and axis labels
# Dates to the month; the chapter gives some, and the rest are from news coverage of each event.
EVENTS = [
    ("2009-10", "GeoCities closes", "erosion"),
    ("2018-04", "Facebook restricts the Graph API after Cambridge Analytica", "enclosure"),
    ("2021-08", "Meta disables NYU Ad Observatory accounts", "exemption"),
    ("2021-09", "Social Science One data found to omit about half of U.S. users", "exemption"),
    ("2023-02", "Twitter ends free API access", "enclosure"),
    ("2023-05", "Reddit cuts off Pushshift", "enclosure"),
    ("2023-07", "Reddit's API pricing takes effect", "enclosure"),
    ("2023-07", "Twitter becomes X, breaking a decade of links", "erosion"),
    ("2024-08", "Meta shuts down CrowdTangle", "enclosure"),
]
events = pd.DataFrame(EVENTS, columns=["date", "event", "pressure"])
events["date"] = pd.to_datetime(events["date"])
events = events.sort_values("date").reset_index(drop=True)

COLORS = {"enclosure": "#2a78d6", "exemption": "#eb6834", "erosion": "#1baf7a"}
fig, ax = plt.subplots(figsize=(9, 4))
for pressure, color in COLORS.items():
    these = events[events["pressure"] == pressure]
    ax.scatter(these["date"], these.index, s=60, color=color, label=pressure, zorder=3)
ax.set_yticks(events.index, events["event"])
ax.invert_yaxis()
ax.grid(axis="x", color="0.9")
ax.legend(title="Pressure", loc="lower left")
ax.set_title("Changes in access to web data named in chapter 3")
ax.set_xlabel("Date (to the month)")
ax.set_ylabel("Event")
plt.tight_layout()
plt.show()
```



💬 Discuss

- Pushshift answers in JSON and CrowdTangle doesn't answer at all. For a researcher who needs their data, which is the worse news, and why?
- On the timeline, when do the enclosure events bunch up? Does that fit the chapter's explanation of why enclosure accelerated when it did?
- Week 2 told the Cambridge Analytica story as one researcher's misuse of the Graph API. On the timeline it's an enclosure event for everyone. Who paid for it?

Step 5 — What the archive remembers (10 minutes)

For an endpoint that's gone, the only evidence left is archival. The Wayback Machine's availability API takes a `url` and returns the closest capture it holds, or none. Write the chapter's `last_snapshot(url)`, which returns the closest capture's timestamp, and ask it about Pushshift's and CrowdTangle's addresses. The archive is under constant load, and answers 429 when an address has asked too often (a classroom on one campus network is one address), so the function treats a 429 as a result rather than a bug. Pause a second between the two requests. See ”Dead-Endpoint Forensics”.

```
# Define last_snapshot(url): request https://archive.org/wayback/available with params={"url":
 ↪  url},
#   HEADERS, and timeout=30; on a 429 return "rate limited — wait and retry"; otherwise return
 ↪  the
#   timestamp of ["archived_snapshots"]["closest"] in the JSON, or "no capture found" if there is
 ↪  none
# Print it for api.pushshift.io and www.crowdtangle.com, with time.sleep(1) between them
def last_snapshot(url):
    """Find the closest archived capture of a page."""
    response = requests.get("https://archive.org/wayback/available",
                            params={"url": url}, headers=HEADERS, timeout=30)
    if response.status_code == 429:
        return "rate limited — wait and retry"
    snapshot = response.json().get("archived_snapshots", {}).get("closest")
    return snapshot["timestamp"] if snapshot else "no capture found"


for page in ["api.pushshift.io", "www.crowdtangle.com"]:
    print(f"{page:<24} {last_snapshot(page)}")
    time.sleep(1)
```

⏱️ Microsprint (5 minutes, alone or in pairs)

Do: the availability API also takes a `timestamp` (`YYYYMMDD`) and returns the capture closest to it. The chapter says the archive kept visiting www.crowdtangle.com until December 2024, but that every capture after 14 August 2024 "records only a redirect". Ask for the capture closest to 1 January 2024, 1 August 2024, 1 December 2024, and 1 June 2025, and print each capture's timestamp and `status`. **Share:** the four captures you got back, and whether the archive ever handed you one from after 14 August 2024.

```
# Microsprint: for each date in 20240101, 20240801, 20241201, 20250601, ask the availability API
 ↪  for
#   www.crowdtangle.com with params {"url": ..., "timestamp": date}; pause 2 seconds between
 ↪  requests
# Print the date asked for and the closest capture's timestamp and status (or the response's
 ↪  status code if not 200)
for date in ["20240101", "20240801", "20241201", "20250601"]:
    response = requests.get("https://archive.org/wayback/available",
                            params={"url": "www.crowdtangle.com", "timestamp": date},
                            headers=HEADERS, timeout=30)
    if response.status_code != 200:
        print(date, "->", response.status_code)
    else:
        closest = response.json().get("archived_snapshots", {}).get("closest")
        print(date, "->", (closest["timestamp"], closest["status"]) if closest else "no capture
         ↪  found")
    time.sleep(2)
```

💬 Discuss

- The chapter reads these timestamps as the shutdowns themselves, written in the archival record. What else could make a site's last capture fall on a given day?
- Week 1 saved pageviews to a CSV because the data may change. What does the archive do that our CSV can't, and what can our CSV do that the archive can't?
- The chapter calls the Wayback Machine counter-erosion infrastructure. Which of the three counter-values does it embody, and what happens to that value if the archive itself closes?

Step 6 — No single Mastodon (14 minutes)

Federated platforms, the chapter says, solve some access problems "while complicating basic measurement: there is no single API endpoint for 'all of Mastodon,' because there is no single Mastodon." Let's measure that. Six servers below run the same software and speak the same protocol, but each server's administrators set its terms. Ask each for its description, `/api/v1/instance`, which reports how many accounts it has and comes with the `x-ratelimit-limit` header we saw in Step 2. Then ask each, with no credentials, for one post from its public timeline, and keep only the status code: the post belongs to someone. Plot the servers' sizes, coloured by whether an anonymous client may read their public timeline. See "Erosion" and "Ownership".

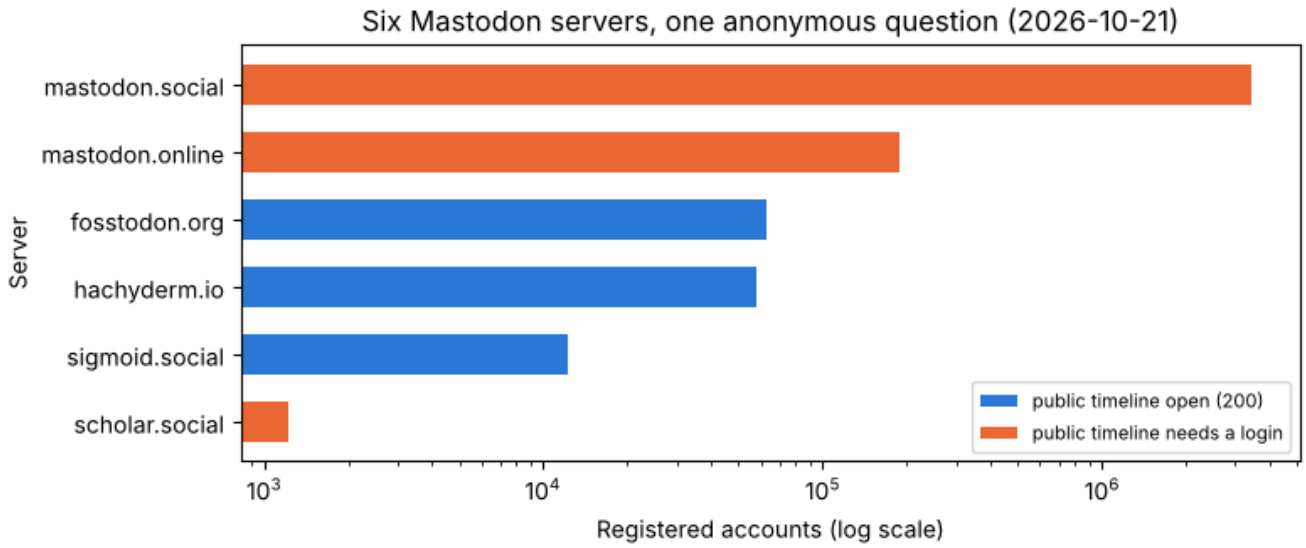
```
# INSTANCES: mastodon.social, mastodon.online, fosstodon.org, hachyderm.io, sigmoid.social,
 ↪  scholar.social
# Define server_terms(domain): get https://{domain}/api/v1/instance; time.sleep(1); then get
#   https://{domain}/api/v1/timelines/public with params={"limit": 1}. Return the domain,
 ↪  ["stats"]["user_count"],
#   the instance's x-ratelimit-limit header, and the public timeline's status code (not its post)
# Make a DataFrame of all six and print it; plot accounts as horizontal bars on a log scale,
#   one colour where the public timeline answered 200 and another where it didn't, with a legend
INSTANCES = ["mastodon.social", "mastodon.online", "fosstodon.org",
             "hachyderm.io", "sigmoid.social", "scholar.social"]
```

```
def server_terms(domain):
    info = requests.get(f"https://{domain}/api/v1/instance", headers=HEADERS, timeout=20)
    info.raise_for_status()
    time.sleep(1)
    public = requests.get(f"https://{domain}/api/v1/timelines/public",
                          params={"limit": 1}, headers=HEADERS, timeout=20)
    time.sleep(1)
    return {"server": domain,
            "accounts": info.json()["stats"]["user_count"],
            "rate_limit": info.headers.get("x-ratelimit-limit"),
            "public_timeline": public.status_code}


servers = pd.DataFrame([server_terms(domain) for domain in INSTANCES])
servers = servers.sort_values("accounts").reset_index(drop=True)
print(servers.to_string(index=False))

open_timeline = servers["public_timeline"] == 200
fig, ax = plt.subplots(figsize=(8, 3.5))
ax.barh(servers.index[open_timeline], servers["accounts"][open_timeline], height=0.6,
        color="#2a78d6", label="public timeline open (200)")
ax.barh(servers.index[~open_timeline], servers["accounts"][~open_timeline], height=0.6,
        color="#eb6834", label="public timeline needs a login")
ax.set_yticks(servers.index, servers["server"])
ax.set_xscale("log")
ax.set_title(f"Six Mastodon servers, one anonymous question ({AUDIT_DATE[:10]})")
ax.set_xlabel("Registered accounts (log scale)")
ax.set_ylabel("Server")
ax.legend(loc="lower right", fontsize=8)
plt.tight_layout()
plt.show()
```

```
         server  accounts rate_limit  public_timeline
 scholar.social      1227        300              422
 sigmoid.social     12243        300              200
   hachyderm.io     57927        300              200
  fosstodon.org     62709        300              200
mastodon.online    188256        300              422
mastodon.social   3442968        300              422
```



⏱️ Microsprint (5 minutes, alone or in pairs)

Do: a short version of Additional Exercise 4. Pick a project that depends on one of today's open rows (Wikipedia, Hacker News, arXiv, Mastodon, or Bluesky). Suppose it closes halfway through your study. From the chapter's "Build" list (research access programs, data donation, independent archives, public records requests, crowdsourcing, shared research infrastructure), choose three backups, and for each write what it trades away: comprehensiveness, reliability, or resolution. **Share:** your source, your first backup, and what it costs you.

```
# Microsprint: your project and its source; three backups from the chapter's "Build" list, each
 ↪  with what it trades away
# Project: how six Mastodon servers' moderation rules changed over a semester (source: each
 ↪  server's public API)
# 1. Independent archive: the Wayback Machine's captures of each server's /about page. Trades
 ↪  away
#    resolution: captures come when the crawler visits, not every week, and a page may have none.
# 2. Data donation: ask each server's administrators for their rule history. Trades away
#    comprehensiveness: only the servers that agree are in the study.
# 3. Shared research infrastructure: start our own weekly collection now and deposit it (week 1's
 ↪  CSV, dated).
#    Trades away reliability only if we stop; it can't recover the weeks before we started.
```

💬 Discuss

- Same software, same protocol, different answers. Whose decision is "this server's public timeline needs a login", and is it enclosure in the chapter's sense?
- If a paper says "we sampled posts from Mastodon", what must its methods section say for a reader to know what was observed and what couldn't be?
- The chapter pairs ownership against erosion. Which of your microsprint's backups is held by a community, and which could one organization retire?

Step 7 — What did we learn? (5 minutes)

In a few sentences, as comments or in a Markdown cell:

- How many of the eight endpoints were open when we ran the audit, and which rows differed from the chapter's 28 August table? Was each difference the platform, or us?
- Which refusals came from the platform itself, and what in `diagnose()` or the identity microsprint showed it?

- Of the three retired endpoints, which is gone, which is gated, and which is repriced? Which leaves someone to negotiate with?
- How many of the six Mastodon servers let an anonymous client read their public timeline? What does that do to the sentence "we collected data from Mastodon"?

Recommended Exercises ahead (in the chapter's companion notebook, due Sunday):

- Step 1, identify yourself: Step 0's `HEADERS`.
- Step 2, probe the OpenAlex anchor by hand: Step 1's single request, read whole, and Step 2's `rate_limit` column, which reads one `X-RateLimit` header.
- Step 3, build the institutional matrix: Step 2's `probe()`, its `Int64` status column, and Step 0's `AUDIT_DATE`.
- Step 4, extend it with two institutions of your own: Step 2's `TARGETS` and `VERDICTS`.
- Step 5, attribute a refusal: Step 3's `diagnose()`, and its microsprint's second identity.
- Step 6, autopsy a retired institutional endpoint: Step 4's `autopsy()` and its categories.
- Step 7, interpret: Step 1 (a refusal can arrive with the wrong content type), Step 4's categories, and Step 6's microsprint on where data goes when a source closes.

```
# Your answers here, as comments
# Four of the eight endpoints were open: Hacker News, arXiv, Mastodon and Bluesky. One row
 ↪  differed from
#   the chapter's 28 August table: Wikipedia answered 429, not 200. That was us, not Wikipedia:
 ↪  the
#   identity microsprint got 403 for the default User-Agent ("Please set a user-agent") and 429
 ↪  for a
#   named one, which is the rate limit on this machine's shared address.
# The platforms' own refusals: Reddit's 403, which diagnose() traced to its own server
 ↪  ("snooserv",
#   behind Varnish) and which both identities got; X's 401 with a JSON problem report, which asks
 ↪  for a
#   token; and Meta Graph's 400.
# Of the retired endpoints, CrowdTangle is gone (its host never answered), Pushshift is gated
 ↪  (403, "Not
#   authenticated"), and Twitter's v1.1 is repriced (400, "Bad Authentication data": it wants a
 ↪  paid
#   token). Pushshift and X leave someone to negotiate with; CrowdTangle doesn't.
# Three of six Mastodon servers let an anonymous client read the public timeline (sigmoid.social,
#   hachyderm.io, fosstodon.org); mastodon.social, mastodon.online and scholar.social answered
 ↪  422.
#   "We collected data from Mastodon" really means "from the servers that let us", and a paper
 ↪  has to
#   name them.
```

Friday: from the lab to the textbook (15 minutes)

The lab ran the chapter's audit again, so it's a good place to look for what has changed or what confused you. Some places to start:

- **Outputs that have changed.** The chapter's tables were recorded on 28 August 2026. Which of our rows differed (Step 2's microsprint), and does the chapter's text around them still hold?
- **Code that broke, or nearly did.** `probe()` and `autopsy()` catch a request that never completes; `diagnose()` doesn't. What happens if you diagnose CrowdTangle's address, and should the chapter say so?
- **A sentence to check.** Step 5's microsprint tested the chapter's claim about CrowdTangle's later captures. Did the availability API show you what the chapter describes?
- **A microsprint that could be an exercise.** Step 6 measured, in a few lines, a claim the chapter makes in one sentence. Would it make a good Additional Exercise?

See `handouts/common/revision-framework.md` for how to propose a revision, as an issue or a pull request.